Übungsblatt 9: Wrap-Up & Finale (State Machine)

Mikrocomputertechnik 1

Zielsetzung

In diesem letzten Übungsblatt schließt sich der Kreis. Sie wiederholen die Werkzeugkette (Toolchain) und die Übersetzungsmuster von C zu Assembler. Anschließend nutzen Sie Ihre Kenntnisse in I/O, Kontrollfluss und Logik, um im Ripes-Simulator ein interaktives Mini-Projekt (Code-Schloss) als Endliche Zustandsmaschine (FSM) zu programmieren.

Aufgabe 1: Die Toolchain

Wenn Sie in der Industrie ein C-Programm mit `gcc main.c` kompilieren, ruft das System im Hintergrund vier verkettete Werkzeuge auf.

Die vier Werkzeuge

Benennen Sie diese vier Werkzeuge in der korrekten Reihenfolge.

Aufgabe des Linkers

Welche spezifische Aufgabe übernimmt das letzte Werkzeug in dieser Kette? Warum ist das Programm nach Schritt 3 (Object-File `.o`) noch nicht ausführbar?

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Aufgabe 2: C-to-Assembly (Klausurvorbereitung)

In der Klausur wird erwartet, dass Sie typische C-Kontrollstrukturen in RISC-V-Assembler übersetzen können. Übersetzen Sie die folgende `while`-Schleife.

Hinweis: Variable `i` liegt in `s0`. Nutzen Sie das invertierte Bedingungsmuster (`bge`), wie es ein Compiler typischerweise tut.

```
int i = 0;
while (i < 10) {
    i++;
}
```

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)

Aufgabe 3: Pipelining und Hazards (Klausurvorbereitung)

Betrachten Sie das folgende Code-Snippet in der 5-stufigen RISC-V-Pipeline:

```
lw t0, 0(a0)
add t1, t0, t0
```

Hazard-Typ

Welches Hazard-Problem entsteht bei dieser Befehlsfolge?

Forwarding vs. Stall

Kann dieses Problem allein durch Forwarding (Bypassing) ohne Zeitverlust gelöst werden? Erklären Sie, wie die Hardware im Pipeline-Raster (IF, ID, EX, MEM, WB) darauf reagieren muss.

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Aufgabe 4: Finale — State Machine (Code-Schloss)

Bauen Sie in Ripes eine Endliche Zustandsmaschine (FSM) als Code-Schloss. Fügen Sie im I/O-Tab ein **D-Pad** (typisch `0xF0000000`) und eine **LED Matrix** (typisch `0xF0000020`) hinzu. Basisadressen unten links ablesen.

FSM-Logik

- Zustandsvariable: Register `s0` für den aktuellen Zustand (0, 1, 2 oder 3).
- Geheimcode: Tasten in der Reihenfolge **OBEN** → **UNTEN** → **RECHTS**.

Verhalten:

Zustand	LED-Farbe	Hex	Erwartete Taste	Sonst
0	Rot	`0x00FF0000`	OBEN → 1	bleiben
1	Gelb	`0x00FFFF00`	UNTEN → 2	zurück auf 0
2	Blau	`0x000000FF`	RECHTS → 3	zurück auf 0
3	Grün	`0x0000FF00`	(offen)	bleibt für immer

D-Pad in Ripes (kein Bitmasken-Modell): Jede Richtung ist ein **eigenes 32-Bit-Register** (LSB = gedrückt):

Richtung	Offset vom D-Pad-Basis
OBEN (UP)	`+0`
UNTEN (DOWN)	`+4`
LINKS (LEFT)	`+8`
RECHTS (RIGHT)	`+12`

Entprellung / Release: Nach erkanntem Tastendruck **warten, bis alle Richtungen wieder losgelassen sind** (Register = 0). Eine reine Delay-Schleife reicht nicht — sonst wertet der Folgezustand denselben Klick als falsche Taste.

Ihre Lösung

(Fügen Sie Ihren Assembler-Code für Ripes hier ein.)